

Latent Preference Modeling for Cross-Session Personalized Tool Calling

Anonymous authors

Paper under double-blind review

Abstract

Users often omit essential details in their requests to LLM-based agents, resulting in under-specified inputs for tool use. This poses a fundamental challenge for tool-augmented agents, as API execution typically requires complete arguments, highlighting the need for personalized tool calling. To study this problem, we introduce **MPT**, a benchmark comprising 265 multi-session dialogues that cover three challenges: Preference Recall, Preference Induction, and Preference Transfer. We also propose **PREFINE**, a test-time memory-augmented method that represents user preferences as evolving hypotheses. Through a generate–verify–refine loop, it extracts reusable constraints from history and improves tool-calling accuracy while using only 1.24% of the tokens required by full-history prompting. These results indicate that robust personalization in agentic systems depends on memory that captures the *reasons* behind user choices, not just the choices themselves.

1 Introduction

LLM-based agents increasingly rely on external tools to execute complex tasks, such as deep research (Xu & Peng, 2025) and computer use (Sager et al., 2026). In practice, users often omit essential details in their requests, making it challenging for agents to interact with tools that require fully specified arguments. To address this, a natural approach is to infer missing information from past user behavior, a central focus of **personalized tool calling** (Moghe et al., 2024; Xu et al., 2025).

Figure 1 illustrates an intuitive example: consider a user who consistently chooses low-cost restaurants, free-entry attractions, and compact rental cars over prior sessions. When she later states, “Book a flight for my trip”, the agent should default to `flight_class=Economy` in the absence of explicit instructions. This goes beyond retrieving similar past actions, requiring modeling a **latent preference**: an implicit, persistent constraint on decision-making derived from recurring behavioral patterns.

We argue that personalized tool calling is not merely a memory retrieval problem, but a reasoning problem centered on latent user-level constraints shaped by interactions over multiple sessions. Prior

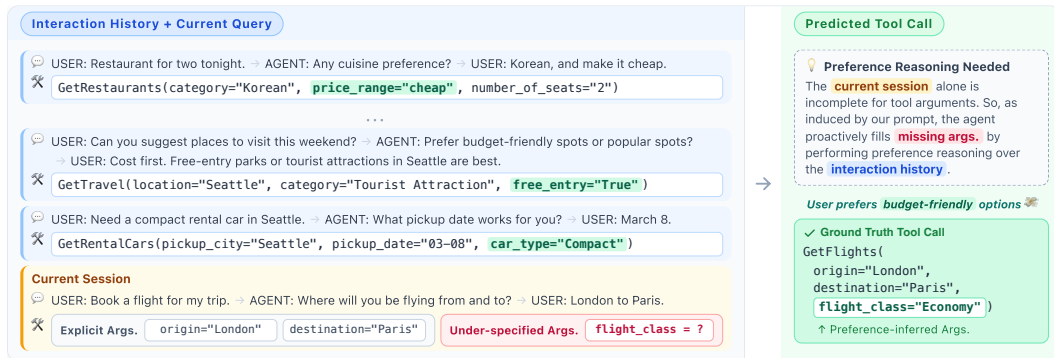


Figure 1: Example of latent preference modeling for personalized tool calling. The agent predicts `flight_class="Economy"` not from the current session, but from prior interactions, where the user consistently selects budget-friendly options in different contexts.

work (Schick et al., 2023; Huang et al., 2025) assumes that user preferences are directly available as user profiles, predefined instructions, or repeated actions within restricted cases. This setting is unrealistic: current agents are rarely provided with profiles and should operate across diverse tasks rather than within limited domains. A key capability of personalized tool-calling agents should thus be to reason over interaction history to capture implicit preferences distributed across noisy, unordered sessions. However, no existing benchmark focuses on this aspect, motivating a dedicated evaluation.

To this end, we propose **Multi-Session Personalized Tool Calling (MPT)**, a benchmark for evaluating personalized tool calling under multi-session interaction histories with intentionally under-specified API arguments. MPT introduces three challenges: (1) *Preference Recall* (direct reuse), (2) *Preference Induction* (aggregating cross-session evidence), and (3) *Preference Transfer* (generalizing to new domains). This taxonomy reveals a performance gap in predicting missing API arguments: models strong on Preference Recall via naïve reuse of prior decisions struggle on Induction and Transfer.

We also present **PREFINE**, a lightweight test-time memory-augmented method that incrementally refines latent user preferences from multi-session interactions via a generate–verify–refine loop. These latent preferences serve as action-level constraints for tool calling and remain effective under changing tool schemas. Experiments show that retrieval-oriented baselines perform well on Preference Recall but degrade sharply on Induction and Transfer, whereas PREFINE improves tool-calling accuracy using only 1.24% of the tokens required by full-history prompting. These results indicate that robust personalization depends on capturing the *reasons* behind user choices, not just the choices themselves.

2 Related Work

Personalized Tool Calling. Recent benchmarks for tool use evaluate an agent’s ability to invoke external APIs in multi-turn settings (Wang et al., 2024; Lee et al., 2024; Yao et al., 2025; Patil et al., 2025; Chakraborty et al., 2025), focusing on planning, failure recovery, and chained execution (Shim et al., 2025). However, they treat tool use as a decision process based solely on the current dialogue state, ignoring prior interactions. In contrast, some studies incorporate dialogue history, regarding user preferences as constraints on API arguments (Moghe et al., 2024; Xu et al., 2025). These approaches assume preferences are explicitly available (e.g., profiles or past API calls), leaving the more realistic setting—where preferences are implicit and must be inferred from interaction history—unaddressed.

Latent Preference Modeling. A related line of research investigates how user preferences can be derived from interaction histories. In dialogue systems, methods such as PrefEval (Zhao et al., 2025), CUPID (Kim et al., 2025), and PersonaMem (Jiang et al., 2025a;b) learn user-specific preference representations for personalization. Conversational recommender systems similarly model user preferences as latent variables from behavioral signals across interactions, enabling generalization beyond observed choices (Luo et al., 2020; Chen et al., 2019; Zhou et al., 2020; Li et al., 2025).

Our notion of **latent preference** is similar in spirit to prior work, as it concerns user-level regularities that are not explicitly stated. However, prior work typically treats such preferences as internal representations for scoring responses or items. In contrast, our setting requires latent preferences to be externalized as reusable textual constraints governing unspecified API arguments.

Memory for Long-Horizon Agents. Research on agentic memory studies how agents maintain, retrieve, and update information over long horizons (Park et al., 2023; Packer et al., 2024; Shinn et al., 2023a; Wang et al., 2023; Wei et al., 2025), with work on multi-session collaboration (Mehri et al., 2026; He et al., 2026), memory compression (Liu et al., 2025), and memory policies (Zhou et al., 2025; Wang et al., 2026). Prior work (Kwon et al., 2023; Zhang et al., 2025) typically stores explicit, factual information in memory, whereas we abstract behavioral patterns into latent preferences.

3 Problem Definition

3.1 Task Definition

Let $S = \{s_1, \dots, s_T\}$ denote a sequence of past dialogue sessions between a single user and an AI agent, where each session s_t consists of a multi-turn dialogue in which the agent may execute one or more API calls. Let $\mathcal{A}_{\leq T}$ denote the accumulated API call list from past sessions S , preserving the

raw executed tool invocations and their argument values across sessions. At timestep $T+1$, the agent observes the current query context q —the sequence of user–agent turns in the current session up to the API decision point. The query may explicitly specify some API arguments while leaving others underspecified. The agent must output an API call a^* that satisfies all explicitly stated constraints in q and infer the remaining preference-driven arguments from latent preferences reflected in the interaction history $(S, \mathcal{A}_{\leq T})$:

$$a^* = \arg \max_{a \in \mathcal{A}} f_{\theta}(a \mid q, S, \mathcal{A}_{\leq T}),$$

where f_{θ} is an LLM-based decision function and $\mathcal{A}$ is the set of valid API call instantiations. Because the action space is bounded by predefined API schemas, our goal is not open-ended preference discovery but *schema-aligned preference reasoning*: identifying persistent argument-level constraints predictable from recurring behavioral patterns in interaction history.

3.2 Preference Modeling Types

Difficulty in latent preference modeling depends on how evidence for a missing argument is distributed across the interaction history, leading to three distinct cases (see Figure 7 in Appendix A.5 for details).

Preference Recall. The history contains clear recurring choices for the same argument–value pair within the same domain (e.g., repeatedly selecting `GetFlights(flight_class=Economy)`). In this case, the missing arguments can often be resolved by retrieving and reusing past choices.

Preference Induction. In this configuration, the missing argument cannot be determined by direct reuse. The agent must aggregate behavioral evidence across interactions spanning tasks and domains. It then predicts a latent preference and instantiates it as concrete argument values.

Preference Transfer. In this setting, the missing argument lacks in-domain evidence. The agent must apply a latent preference from other domains to guide argument selection in the target domain.

4 Dataset Construction: MPT

To evaluate latent preference modeling (§3), we introduce **Multi-Session Personalized Tool-Calling (MPT)**,¹ a benchmark pairing multi-session interaction histories with queries featuring intentionally under-specified API arguments. Each instance is designed to reflect one of the three problem types in §3.2. We build MPT on top of Schema-Guided Dialogue (SGD; Rastogi et al. (2020)), a multi-domain task-oriented dialogue corpus with API schemas.² SGD provides structured domain–slot–value representations grounding preferences at the level of executable tool arguments, and spans semantically related domains that naturally support the study of cross-domain preference consistency.

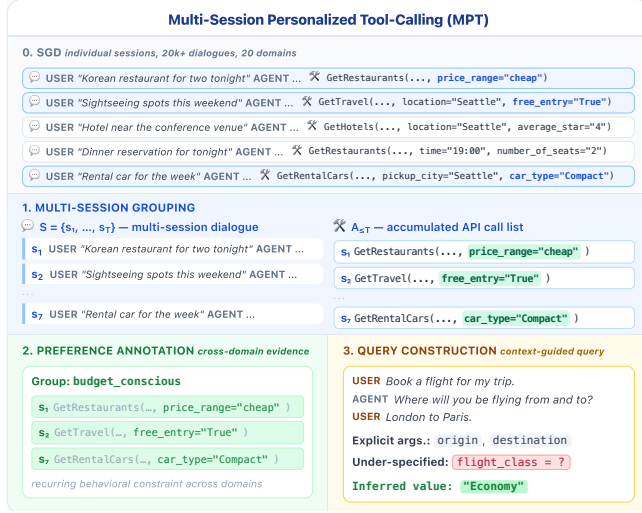
Multi-Session Grouping. MPT is constructed in three stages, as illustrated in Figure 2. We first group multiple SGD sessions into a single multi-session dialogue $S = \{s_1, \dots, s_T\}$ for one user. Preference signals emerge from repeated argument patterns within a domain and consistent cross-domain selection behavior, neither of which is fully captured in a single session s_t . For each dialogue, we accumulate per-session API calls into an *API call list* $\mathcal{A}_{\leq T}$, preserving the action trace from which latent preferences can be inferred. Together, S and $\mathcal{A}_{\leq T}$ constitute the interaction history $(S, \mathcal{A}_{\leq T})$.

Preference Annotation. In the second phase, we enable evaluation of latent preference modeling by manually grouping related API arguments into higher-level preference categories. Since SGD provides only domain–slot–value triples without preference labels, this process is key to assigning gold-standard annotations. Formally, a preference group consists of a set of preferences, each spanning diverse but related API–argument pairs. For instance, the BUDGET group contains two preferences: *low_cost* and *high_cost*. The *low_cost* preference covers API–argument pairs such as `GetRestaurants(price_range="cheap")` and `GetTravel(free_entry=True)`. The full mapping table for 58 API–argument pairs is provided in Appendix A.2.

¹We will publicly release MPT and experiment codes to support future research.

²We keep the original schema definitions and apply schema normalization following the prior work (Moghe et al., 2024), enabling coherent multi-session histories while remaining compatible with the tool schemas. See Appendix A.1 for the full schema.

We validate the grouping annotations through a human study with 19 annotators (Appendix A.6), finding strong agreement for both budget (89.7%) and travel groups (97.4%), confirming that our preference groups reflect commonsense. Because the groups are defined at the level of behavioral constraints rather than specific slot names, the scheme generalizes to tool calling with different schemas. In other words, any new API exposing cost- or party-size-related arguments falls under the same taxonomy without redefinition, making the scheme broadly applicable beyond SGD, as verified in §7.5.



Query Construction. For each domain, we manually design query templates that omit one or more preference-related arguments.³ Each MPT instance combines a target-domain query q with an interaction history $(S, A_{S,T})$.

Figure 2: Overview of MPT construction. Individual SGD sessions are grouped into a multi-session interaction history $(S, A_{S,T})$, from which cross-domain preference evidence is annotated as shared behavioral constraints. Target queries are constructed by intentionally under-specifying preference-sensitive arguments.

We design two query types. **Context-guided queries** include in-session dialogue context that partially states explicit argument constraints. **Context-free queries** omit such information entirely, requiring the agent to rely solely on preference modeling to fill in missing arguments. This distinction lets us evaluate both preference-driven argument completion under partial in-session specification and preference modeling when the current query provides little guidance. We refer readers to Table 7 in Appendix A.5 for example queries.

The preference modeling type for each instance is determined not by q alone but by the relationship between q and $(S, A_{S,T})$ (Figure 1), allowing the same history to be paired with multiple queries.

Dataset Statistics. MPT comprises 265 multi-session dialogues with 2,020 sessions and 39,884 turns, averaging 7.6 sessions per dialogue and 19.7 turns per session. It includes 332 Preference Recall instances, 293 Induction instances, and 472 Transfer instances (see Table 6 in Appendix A.3 and Appendix A.4 for details). Preference-relevant evidence is distributed across sessions rather than concentrated within any single interaction, making MPT particularly suited for distinguishing shallow action reuse from preference reasoning that requires abstraction over long interaction histories.

5 Proposed Method: PREFINE

5.1 Motivation

A straightforward approach to personalized tool calling is to provide the full interaction history and let an LLM complete under-specified arguments end-to-end. However, access to long histories does not reveal which past decisions reflect reusable user constraints and which are merely local or situational. As shown in §7.1, LLMs given full history often fail to abstract and apply the behavioral regularities needed to complete such decisions. This implies that personalized tool calling is not primarily a retrieval problem but an abstraction problem: the model must infer reusable constraints from repeated behavior and apply them to guide future argument selection.

³We exclude time and location arguments in our setting as they are not directly related to user preferences.

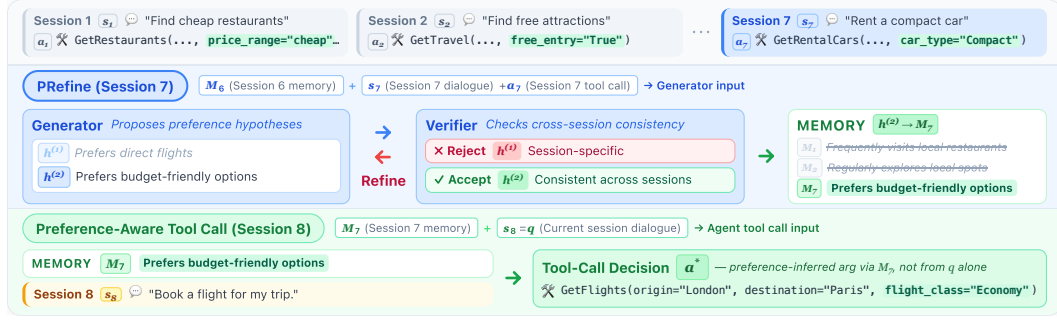


Figure 3: PREFINE’s generate-verify-refine loop. At each session $T+1$ (e.g., Session 7), candidate preference hypotheses $h^{(i)}$ are generated from the current dialogue s_{T+1} , tool call a_{T+1} , and prior memory M_T (e.g., M_6). Here, M_T denotes the single preference hypothesis accepted at session T and is updated to M_{T+1} upon acceptance of a new hypothesis. The updated memory is then used to constrain tool-call decisions a^* in subsequent sessions (e.g., Session 8).

5.2 Latent Preference as Hypotheses

We view a **latent preference** as an implicit, persistent constraint on API argument selection expressed through recurring behavioral patterns. Such preferences may appear as repeated in-domain behavior (e.g., repeated selections of `flight_class="Economy"`) or as cross-domain regularities (e.g., consistently choosing budget-oriented options across flights, restaurants, and hotels). Because they are not directly observed and often emerge only from evidence accumulated across sessions, latent preferences must be treated as **hypotheses**. A plausible hypothesis at one point may later become too narrow or contradicted by new evidence; preference modeling is therefore not a one-shot prediction, but an ongoing process of maintaining and updating beliefs about the user’s latent constraints.

5.3 PREFINE: A Memory-Based System for Latent Preference Refinement

As latent preferences are not directly observed, are only partially identified at each session, and may require revision as new evidence arrives, preference memory cannot be treated as a static store of episodes. Instead, it should function as a *revisable hypothesis of preference constraints*. PREFINE embodies this philosophy, storing the current best abstraction of behavioral regularities supported by accumulated evidence and applicable to future tool use. Table 1 highlights the unique characteristics of PREFINE from multiple perspectives, including memory content and update mechanisms.

Method	Memory Content	Update Mechanism	Actionable	Latent Preference-Aware
RAG	Raw utterances	Static index	✗	✗
Mem0	Extracted facts	Append/overwrite	✗	✗
LangMem	Structured facts	LLM rewrite	✗	✗
PREFINE	Latent constraints	Generate-verify-refine	✓	✓

Table 1: Comparison of memory-augmented methods in terms of content and memory update mechanisms. PREFINE is the only method that stores latent preferences and refines them iteratively.

Generate-Verify-Refine Loop. As shown in Figure 3, at session $T+1$, PREFINE takes as input the current dialogue s_{T+1} , the executed API call(s), and the prior memory M_T —the single preference hypothesis accepted at session T . We implement the update from M_T to M_{T+1} as a generate-verify-refine loop, following self-refinement algorithms (Madaan et al., 2023; Shinn et al., 2023b). This design is motivated by the nature of latent preferences: no single session fully determines the underlying constraint, and subsequent sessions may refine, broaden, or overturn earlier hypotheses.

Specifically, a **generator** proposes candidate preference hypotheses ($h^{(1)}, h^{(2)}, \dots$) that explain the observed user actions at a more abstract level. A **verifier** then evaluates whether each candidate is admissible as preference memory under four validity conditions:⁴ (1) *Evidence Support*, whether the hypothesis is grounded in multiple or mutually consistent interactions; (2) *Abstraction Quality*,

⁴Detailed rubrics and prompts are provided in Appendix D.

Session	a_t	Step	Hypothesis (Generate, Refine) / Verdict (Verify)
s_1	a_1 : GetMovies(average_rating = 6);	Generate	User prefers moderately rated movies.
		Verify	[REJECT] Over-specific and unsupported abstraction.
		Refine	User prefers accessible movie content.
		Verify	[REJECT] Insufficient evidence for future decisions.
		Refine	User has minimal interest in movies.
		Verify	[PASS] Abstract and observation-supported.
s_2	a_2 : GetWeather(city = San Francisco);	Generate	User prefers movies while engaging with other domains.
		Verify	[REJECT] Failed to account for weather-domain interaction.
		Refine	User prioritizes movies but engages across domains.
		Verify	[PASS] Cross-domain flexibility ensured.
s_3	a_3 : GetRentalCars(car_type = Standard), GetRestaurants(price_range = Cheap);	Generate	User prefers economical and simple options across domains.
		Verify	[PASS] Consistent cross-domain behavioral signal.
s_4	a_4 : GetHotels(average_star = 1);	Generate	User prefers budget-friendly and simple interactions.
		Verify	[PASS] Stable and memory-worthy preference.
M_4		Budget-conscious and simple interaction style.	
[Inference Example] q : "I'd like to book a flight." $\rightarrow a^*$: GetFlights(flight_class = Economy)			

Table 2: Example of preference modeling with PREFINE via the generate-verify-refine loop, where the verifier rejects over-specific hypotheses and retains generalizable abstractions.

whether it generalizes beyond a one-off event or a slot-level restatement; (3) *Actionability*, whether it can meaningfully bias or constrain future API argument selection; and (4) *Temporal Consistency*, whether it remains compatible with the most recent stable behavioral pattern. Otherwise, weak or narrow hypotheses are returned to the generator for **refinement**, where they are revised based on the verifier’s feedback. Table 2 illustrates how this process progressively transforms narrow session-level hypotheses into reusable cross-domain constraints as observations accumulate across sessions.

Schema-Agnostic Preference Memory. A key property of PREFINE is that its memory is schema-agnostic. Rather than storing schema-specific API signatures, PREFINE retains abstract preference constraints that are usable for different tool interfaces. At session $T+1$, the inference model conditions on the current query q together with the retained memory M_T , and grounds these abstract constraints to the API schema at test time. Because schema grounding is deferred to inference, memory built under one schema remains useful even when the test-time schema differs in slot names, argument inventories, or schema realizations. We evaluate this property in §7.5, with detailed dynamic-schema examples provided in Appendix C.6.

6 Experimental Setup

Methods and Models. We evaluate all methods without additional training to assess test-time latent preference modeling for personalized tool calling. We compare PREFINE against four baselines: Base prompting, RAG (Lewis et al., 2020), Mem0 (Chhikara et al., 2025), and LangMem (LangChain AI, 2025), representing full-dialogue prompting, retrieval-based memory, summary-based memory, and agentic memory, respectively. Under Base prompting, the inference LLM receives full dialogue history together with the accumulated API list. RAG, Mem0, LangMem, and PREFINE replace the full dialogue history with method-specific memory, while keeping the same cumulated API list. Detailed experimental settings and model nomenclature are provided in Appendix B.

To test the robustness of PREFINE to the choice of memory-construction model, we build individual preference memories with four base LLMs⁵ and evaluate them with the eight inference LLMs reported in Table 3, covering all 4×8 memory-inference model combinations. Table 3 reports the performance averaged over the four memory-construction models, for each inference LLM. For RAG, Mem0, and LangMem, we report only the best-performing backbone (Gemini-3-Flash) in Table 3, deferring the rest to Appendix C.4. For PREFINE, we cap the generate-verify-refine loop at three iterations. As shown in Appendix C.5, increasing the budget to ten iterations provides no consistent gain, despite higher inference cost, consistent with prior findings on iterative refinement in LLMs (Madaan et al., 2023; Huang et al., 2023).

⁵Gemma-3-12B-IT, GPT-4o-mini, R1-Distill-Llama-8B, R1-Distill-Qwen-7B

Base LLM	Context-Guided Query										Context-Free Query									
	Pref. Recall			Pref. Induction			Pref. Transfer			Avg. OA-F1	Pref. Recall			Pref. Induction			Pref. Transfer			Avg. F1
	P-EM	EA-F1	OA-F1	P-EM	EA-F1	OA-F1	P-EM	EA-F1	OA-F1		Prec.	Rec.	F1	Prec.	Rec.	F1	Prec.	Rec.	F1	
BASE PROMPTING: Full-dialogue context																				
CodeGemma-7B	18.67	38.88	38.17	4.10	32.78	30.35	0.64	37.19	29.37	32.63	19.63	67.31	30.39	12.53	54.27	20.36	5.00	15.04	7.50	19.42
Gemma-3-12B	7.23	60.36	49.49	2.73	57.64	48.16	0.00	55.86	46.22	46.95	47.78	38.78	42.81	43.24	38.23	40.58	13.65	8.47	10.46	32.66
R1-Distill-Llama-8B	34.94	65.12	61.03	18.43	62.60	58.02	6.14	59.37	49.57	56.21	32.29	71.47	44.48	25.24	70.65	37.20	8.13	18.01	11.21	30.96
R1-Distill-Qwen-7B	13.55	33.49	31.58	7.17	27.88	25.50	0.64	25.87	20.12	25.73	21.12	56.51	30.75	13.33	44.37	20.51	3.10	8.26	4.51	18.59
GPT-4o-mini	32.23	58.21	53.54	18.43	62.46	57.34	4.87	61.98	48.94	53.27	50.09	76.18	60.44	42.39	78.84	55.13	16.10	27.12	20.21	45.26
GPT-5-mini	47.59	65.38	66.69	23.21	63.46	61.78	11.65	61.09	52.25	60.24	61.42	88.64	72.56	44.67	<u>81.57</u>	57.73	19.95	<u>36.02</u>	25.68	51.99
GPT-5	51.20	62.33	64.77	32.42	65.34	64.01	23.94	64.27	55.47	61.42	59.39	86.70	70.50	43.22	76.11	55.13	19.25	31.36	23.85	49.83
Gemini-3-Flash	62.65	72.73	<u>74.25</u>	28.67	69.66	<u>66.49</u>	14.62	69.68	56.54	<u>65.76</u>	63.27	<u>87.81</u>	73.55	44.32	81.23	57.35	22.11	33.69	26.70	52.53
Average	33.51	57.06	54.94	16.89	55.23	51.46	7.81	54.41	44.81		44.37	71.68	53.19	33.62	65.66	43.00	13.41	22.25	16.26	
MEMORY-AUGMENTED METHODS																				
RAG (Top-5)	50.60	69.14	67.99	24.91	67.60	61.34	8.26	69.40	55.88	61.74	52.42	60.11	56.00	45.98	70.31	55.60	21.68	24.58	23.04	44.88
Mem0	31.93	64.59	59.79	27.99	65.52	62.05	16.31	65.93	54.85	58.90	52.36	55.40	53.84	48.51	72.35	58.08	25.59	27.75	26.63	46.18
LangMem	<u>64.40</u>	64.54	67.83	26.62	69.10	63.56	6.57	57.59	46.79	59.40	69.25	86.70	77.00	46.90	67.24	55.26	13.59	12.92	13.25	48.50
PREFINE																				
CodeGemma-7B	<u>59.64</u>	<u>69.50</u>	<u>70.51</u>	16.38	<u>65.86</u>	<u>61.00</u>	1.61	<u>67.20</u>	<u>53.97</u>	61.83	<u>35.40</u>	<u>81.22</u>	<u>49.31</u>	<u>30.51</u>	<u>70.65</u>	<u>40.80</u>	7.41	18.43	10.57	33.56
Gemma-3-12B	20.48	79.28	69.27	5.67	74.11	63.24	0.21	75.38	63.54	65.35	76.10	63.66	69.30	52.10	57.54	54.28	12.67	6.36	8.45	44.01
R1-Distill-Llama-8B	42.05	<u>62.35</u>	61.63	22.12	<u>62.07</u>	58.22	4.83	<u>52.22</u>	<u>42.95</u>	54.27	<u>44.72</u>	<u>71.30</u>	<u>54.95</u>	28.82	<u>60.68</u>	39.08	9.26	13.77	11.07	35.03
R1-Distill-Qwen-7B	<u>32.17</u>	<u>59.05</u>	<u>54.60</u>	17.20	<u>58.93</u>	<u>51.20</u>	3.60	<u>47.38</u>	<u>37.81</u>	47.87	<u>36.00</u>	<u>57.23</u>	<u>44.19</u>	26.69	49.15	34.58	10.88	16.74	13.18	30.65
GPT-4o-mini	<u>49.88</u>	<u>72.65</u>	<u>68.71</u>	28.12	<u>70.73</u>	65.03	9.19	69.97	56.99	63.58	<u>62.11</u>	<u>66.70</u>	64.25	50.22	<u>73.99</u>	59.78	20.92	23.05	21.84	48.62
GPT-5-mini	51.45	68.03	68.08	<u>32.97</u>	67.71	65.16	21.02	67.23	58.47	63.90	<u>73.23</u>	<u>83.43</u>	<u>77.90</u>	<u>53.18</u>	76.72	<u>62.79</u>	<u>29.59</u>	<u>30.00</u>	<u>29.62</u>	<u>56.77</u>
GPT-5	52.41	66.74	67.85	37.95	65.87	64.80	26.19	67.23	59.29	63.98	<u>74.46</u>	<u>82.99</u>	78.41	54.87	<u>70.85</u>	61.81	27.21	28.18	27.64	55.95
Gemini-3-Flash	64.88	<u>72.76</u>	74.75	29.76	69.98	67.17	18.81	<u>70.55</u>	<u>59.62</u>	67.18	<u>71.45</u>	<u>85.37</u>	<u>77.75</u>	51.10	82.05	62.95	30.92	39.87	34.81	58.50
Avg. Gain (%p)	<u>13.11</u>	<u>11.73</u>	<u>11.99</u>	6.88	<u>11.68</u>	<u>10.52</u>	2.87	<u>10.23</u>	<u>9.27</u>		<u>14.81</u>	2.31	<u>11.32</u>	9.82	2.05	9.01	5.20	0.20	3.38	
Average	46.62	68.80	66.92	23.77	66.91	61.98	10.68	64.65	54.08		59.18	73.99	64.51	43.44	67.70	52.01	18.61	22.05	19.65	

Table 3: Performance comparison between baselines and PREFINE under context-guided and context-free query settings. **Bold** indicates the best performance, and underline indicates the second-best performance. Shaded cells indicate performance changes introduced by PREFINE relative to the LLM: **green** denotes gains and **red** denotes losses. The intensity of the shading reflects the magnitude of change ($|\Delta| < 5$: light, $5 \leq |\Delta| < 10$: moderate, $10 \leq |\Delta| < 20$: strong, $|\Delta| \geq 20$: very strong). Exact numerical changes are reported in Appendix C.1.

Metrics. In **context-guided** queries, the model must both extract explicitly stated arguments from the query context and fill in unspecified arguments. We report **Preference Exact Match (P-EM)**, **Explicit-Argument F1 (EA-F1)**, and **Overall-Argument F1 (OA-F1)**. P-EM measures whether the model correctly predicts the preference-driven, yet unspecified arguments. EA-F1 measures tool-calling ability on explicitly specified arguments. OA-F1 evaluates over all arguments. For **context-free** queries, no argument values are explicitly mentioned, so the task isolates preference modeling itself. Here we report **precision**, **recall**, and **F1** over the preference-driven argument completions.

Overall, P-EM and the *context-free* query metrics capture latent preference modeling, EA-F1 reflects standard tool-call generation, and OA-F1 captures how well the model handles both.

7 Experimental Results

7.1 Existing Baselines Recover Observations but Not Latent Preferences

As reported in Table 3, existing baselines are relatively strong in Preference Recall, where the correct action can be predicted by relying on previous actions. But they degrade in Preference Induction and Preference Transfer, where the model must infer and apply a latent constraint. This gap is clearest for Base prompting in the context-free query setting, which isolates preference modeling: average F1 drops from 53.19% in Preference Recall to 43.00% in Preference Induction and 16.26% in Preference Transfer. A similar pattern holds for RAG, Mem0, and LangMem: their context-guided Preference Recall P-EM reaches 50.60%, 31.93%, and 64.40%, respectively, but these gains do not persist in Preference Induction or Preference Transfer, and the same trend appears in the context-free query setting. In summary, these results suggest that existing baselines can support direct behavioral reuse, but not the induction or transfer of latent preferences.

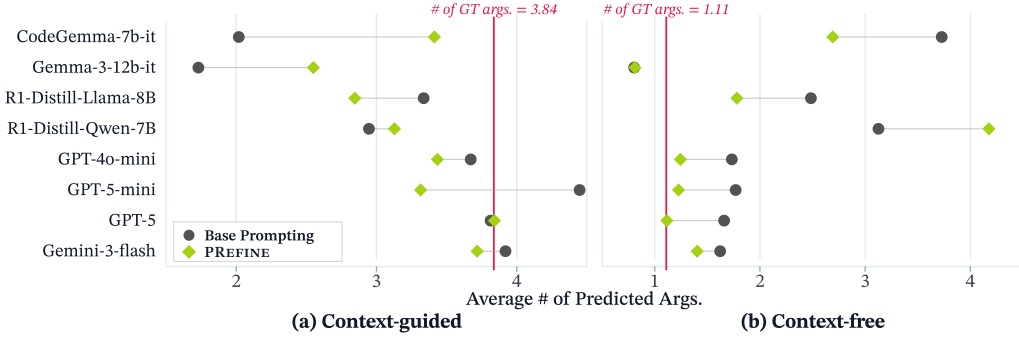


Figure 4: Average number of predicted API arguments per model under Base prompting and PREFINE. Circles denote Base prompting, diamonds denote PREFINE, and the red vertical line marks the average ground-truth number of arguments.

244 7.2 How PREFINE Improves Tool Use

245 **Compact, Verified Latent Preference Memory.** Consistent with our abstraction view in §5, PRE-
 246 FINE improves both preference-driven argument prediction (P-EM) and explicit-argument prediction
 247 (EA-F1) in the context-guided query setting (Table 3, left). We attribute the EA-F1 gain to reduced
 248 test-time dependence on long dialogue history: by providing a compact latent preference memory,
 249 PREFINE lets the inference model focus on interpreting the current request and executing the API
 250 call. We attribute the P-EM gain to the quality of the retained memory itself. As illustrated in Table 2,
 251 the `generate-verify-refine` loop filters out over-specific or weakly supported hypotheses
 252 and retains only verified abstractions that remain supported across sessions and usable for future
 253 tool decisions. This gives the inference model directly applicable preference guidance, rather than
 254 requiring it to rediscover latent constraints from the full interaction history at inference time.

255 **Better-Calibrated Argument Generation.** Beyond slot-level correctness, PREFINE also improves
 256 how well the model selects which schema arguments to instantiate. Although the schema defines
 257 the candidate slots, the model must still decide which subset is warranted by the current query and
 258 inferred preferences. Errors therefore arise not only from predicting incorrect values, but also from
 259 introducing unsupported arguments or omitting required ones. Figure 4 shows that, for most models,
 260 PREFINE predictions lie closer to the red ground-truth line in both query settings. To quantify this
 261 pattern, we compute the mean absolute deviation between each method’s predicted argument count
 262 and the ground-truth number of arguments (# of GT args.). This deviation decreases from 0.77 to
 263 0.56 in the context-guided query setting and from 1.08 to 0.77 in the context-free query setting,
 264 corresponding to reductions of 28.1% and 28.7%, respectively. This indicates better action-space
 265 alignment: by making latent preferences explicit, PREFINE narrows the set of plausible candidate
 266 actions, reducing both unsupported extra arguments and missing required ones.

267 7.3 Memory Efficiency, Scalability, and Utility

268 Figure 5 shows that PREFINE is substantially more compact than full dialogue history and other
 269 memory baselines. Across the dataset, it uses 23.28 tokens on average per dialogue, corresponding to
 270 1.24% of the full dialogue history and more than an 80% reduction relative to the baseline memory
 271 methods. Its footprint also remains nearly constant as sessions accumulate, staying around 20–25
 272 tokens even after ten sessions, suggesting that effective latent personalized tool calling depends more
 273 on retaining compact reusable constraints than on carrying forward long interaction histories.

274 7.4 When and Where PREFINE Helps Most

275 **Preference Transfer Gains Depend on Inference-Time Preference Application.** Preference
 276 Transfer requires both *preference abstraction*, namely inferring a latent preference that generalizes
 277 beyond the observed interaction history, and *preference application*, namely determining when that
 278 preference is relevant in a new context and translating it into argument-level constraints. PREFINE

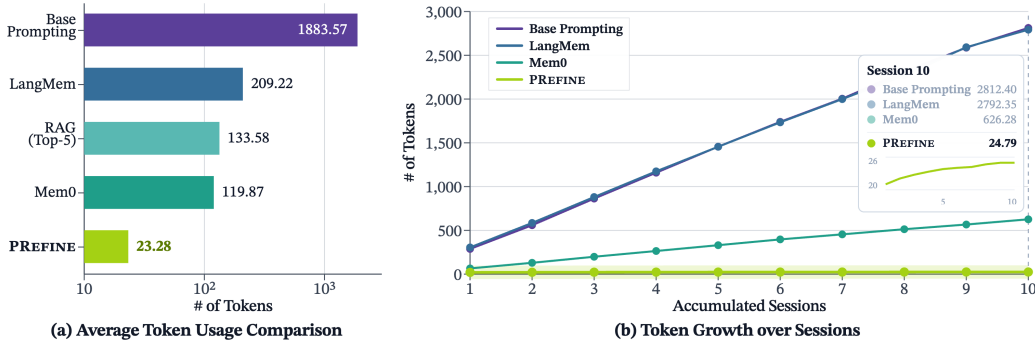


Figure 5: Memory footprint comparison across methods. (a) Average number of retrieved tokens at test time. (b) Memory token growth over accumulated sessions.

supports the former by storing latent preference hypotheses distilled from past interactions and the latter by providing directly usable preference guidance at inference time. As reported in Table 3, this improves Preference Transfer for most inference LLMs. The remaining variation across backbones suggests that, once such memory is available in an actionable form, transfer performance depends on how effectively each model applies the stored preference in a new context.

Action-Space Alignment Introduces Predictable Trade-offs. The same mechanism that makes PREFINE effective—narrowing the action space toward more plausible tool calls—also explains where its gains are smaller. In the context-guided query setting, backbones such as R1-Distill-Llama-8B can become overly conservative: as shown in Figure 4, it already under-generates arguments in the base setting and predicts even fewer after applying PREFINE ($3.34 \rightarrow 2.85$), which results in lower EA-F1 in Table 3. In the context-free query setting, Gemma-3-12B shows little calibration benefit in Figure 4 and correspondingly exhibits a slight drop in Preference Transfer performance in Table 3. More generally, tighter calibration can trade recall for precision when pruned arguments are in fact required, lowering recall across models in Table 3. These cases are therefore best understood not as contradictions to the overall trend, but as predictable trade-offs of stronger action-space control.

7.5 PREFINE Supports Dynamic Schema

In realistic settings, tool interfaces evolve: argument inventories change and new schemas are introduced. Therefore we test whether PREFINE memory built under the original MPT schema remains useful under a dynamic schema. As detailed in Appendix C.6, this evaluation uses unseen API domains whose argument names and values differ from those seen during memory construction.

Even under schema mismatch, PREFINE retains clear gains. With GPT-5, context-guided P-EM rises from 3.75% to 47.00% and context-free F1 from 36.39% to 51.45%; similar gains appear for Gemini-3-Flash (Appendix C.6). This suggests that PREFINE’s abstract preference constraints can be re-grounded to evolving schema at inference time. This supports the claim in § 5.3 that PREFINE is schema-agnostic at the memory level while remaining schema-constrained at execution time.

8 Conclusion

Personalized tool calling often requires more than retrieving past actions: it requires inferring latent user constraints from multi-session behavior and applying them to under-specified API arguments. To study this, we present MPT, which provides three challenges—Preference Recall, Preference Induction, and Preference Transfer—and reveals a consistent gap between naïve pattern matching and true latent preference modeling. We also propose PREFINE, a lightweight test-time memory-based method that represents preferences as revisable hypotheses. By generating, verifying, and refining reusable preference constraints, PREFINE improves personalized tool calling and remains effective under dynamic schema. A future avenue is to extend this framework to richer forms of personalization, including broader preference taxonomies, evolving preferences, and noisier long-horizon interactions.

Ethics Statement

This work introduces a benchmark and method for personalized tool calling based on the Schema-Guided Dialogue dataset, which contains no personally identifiable information. The preference annotations were conducted by 19 human annotators who participated voluntarily. Our method is designed to improve agent personalization from behavioral history; while this raises general privacy considerations around user data retention, our benchmark operates entirely on synthetic task-oriented dialogues and does not involve real user data. We release MPT and experiment code to facilitate reproducible research.

References

- Amartya Chakraborty, Paresh Dashore, Nadia Bathaee, Anmol Jain, Anirban Das, Shi-Xiong Zhang, Sambit Sahu, Milind Naphade, and Genta Indra Winata. T1: A tool-oriented conversational dataset for multi-turn agentic planning, 2025. URL <https://arxiv.org/abs/2505.16986>.
- Qibin Chen, Junyang Lin, Yichang Zhang, Ming Ding, Yukuo Cen, Hongxia Yang, and Jie Tang. Towards knowledge-based recommender dialog system. In Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (eds.), *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 1803–1813, Hong Kong, China, November 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1189. URL <https://aclanthology.org/D19-1189/>.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Joseph L. Fleiss. Measuring nominal scale agreement among many raters. *Psychological Bulletin*, 76(5):378–382, 1971. doi: 10.1037/h0031619.
- Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, et al. Memoryarena: Benchmarking agent memory in interdependent multi-session agentic tasks. *arXiv preprint arXiv:2602.16313*, 2026.
- Jiaxin Huang, Shixiang Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. Large language models can self-improve. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 1051–1068, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.67. URL <https://aclanthology.org/2023.emnlp-main.67/>.
- Xu Huang, Yuefeng Huang, Weiwen Liu, Xingshan Zeng, Yasheng Wang, Ruiming Tang, Hong Xie, and Defu Lian. Advancing and benchmarking personalized tool invocation for llms, 2025. URL <https://arxiv.org/abs/2505.04072>.
- Bowen Jiang, Zhuoqun Hao, Young-Min Cho, Bryan Li, Yuan Yuan, Sihao Chen, Lyle Ungar, Camillo J. Taylor, and Dan Roth. Know me, respond to me: Benchmarking llms for dynamic user profiling and personalized responses at scale, 2025a. URL <https://arxiv.org/abs/2504.14225>.
- Bowen Jiang, Yuan Yuan, Maohao Shen, Zhuoqun Hao, Zhangchen Xu, Zichen Chen, Ziyi Liu, Anvesh Rao Vijjini, Jiashu He, Hanchao Yu, Radha Poovendran, Gregory Wornell, Lyle Ungar, Dan Roth, Sihao Chen, and Camillo Jose Taylor. Personamem-v2: Towards personalized intelligence via learning implicit user personas and agentic memory, 2025b. URL <https://arxiv.org/abs/2512.06688>.
- Tae Soo Kim, Yoonjoo Lee, Yoonah Park, Jiho Kim, Young-Ho Kim, and Juho Kim. Cupid: Evaluating personalized and contextualized alignment of llms from interactions. *arXiv preprint arXiv:2508.01674*, 2025.

- Deuksin Kwon, Sunwoo Lee, Ki Hyun Kim, Seojin Lee, Taeyoon Kim, and Eric Davis. What, when, and how to ground: Designing user persona-aware conversational agents for engaging dialogue. In Sunayana Sitaram, Beata Beigman Klebanov, and Jason D Williams (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 5: Industry Track)*, pp. 707–719, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-industry.68. URL <https://aclanthology.org/2023.acl-industry.68/>.
- J. Richard Landis and Gary G. Koch. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174, 1977. doi: 10.2307/2529310.
- LangChain AI. LangMem: Long-term memory SDK for LLM agents. <https://langchain-ai.github.io/langmem/>, 2025. Accessed: 2025-11-20.
- Shinbok Lee, Gaeun Seo, Daniel Lee, Byeongil Ko, Sunghee Jung, and Myeongcheol Shin. Functionchat-bench: Comprehensive evaluation of language models’ generative capabilities in korean tool-use dialogs, 2024. URL <https://arxiv.org/abs/2411.14054>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33: 9459–9474, 2020.
- Guanrong Li, Kuo Tian, Jinnan Qi, Qinghan Fu, Zhen Wu, and Xinyu Dai. Harmonizing large language models with collaborative behavioral signals for conversational recommendation. *arXiv preprint arXiv:2503.10703*, 2025.
- Jiaqi Liu, Yaofeng Su, Peng Xia, Yiyang Zhou, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. Simplemem: Efficient lifelong memory for llm agents. *arXiv preprint arXiv:2601.02553*, 2025. URL <https://github.com/aiming-lab/SimpleMem>.
- Kai Luo, Scott Sanner, Ga Wu, Hanze Li, and Hojin Yang. Latent linear critiquing for conversational recommender systems. In *Proceedings of The Web Conference 2020*, WWW ’20, pp. 2535–2541, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450370233. doi: 10.1145/3366423.3380003. URL <https://doi.org/10.1145/3366423.3380003>.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fblb55a505a9e9f6c0ff3-Abstract-Conference.html.
- Shuhaib Mehri, Priyanka Kargupta, Tal August, and Dilek Hakkani-Tür. Learning user preferences through interaction for long-term collaboration. *arXiv preprint arXiv:2601.02702*, 2026.
- Nikita Moghe, Patrick Xia, Jacob Andreas, Jason Eisner, Benjamin Van Durme, and Harsh Jhamtani. Interpreting user requests in the context of natural language standing instructions. In Kevin Duh, Helena Gomez, and Steven Bethard (eds.), *Findings of the Association for Computational Linguistics: NAACL 2024*, pp. 4043–4060, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-naacl.255. URL <https://aclanthology.org/2024.findings-naacl.255/>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems, 2024. URL <https://arxiv.org/abs/2310.08560>.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior, 2023. URL <https://arxiv.org/abs/2304.03442>.

- Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (bfc1): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025.
- Abhinav Rastogi, Xiaoxue Zang, Srinivas Sunkara, Raghav Gupta, and Pranav Khaitan. Towards scalable multi-domain conversational agents: The schema-guided dialogue dataset, 2020. URL <https://arxiv.org/abs/1909.05855>.
- Pascal J Sager, Benjamin Meyer, Peng Yan, Rebekka von Wartburg-Kottler, Layan Etaiwi, Aref Enayati, Gabriel Nobel, Ahmed Abdulkadir, Benjamin F Grewe, and Thilo Stadelmann. A comprehensive survey of agents for computer use: Foundations, challenges, and future directions. *Journal of Artificial Intelligence Research*, 85, 2026.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html.
- Jeonghoon Shim, Gyuhyeon Seo, Cheongsu Lim, and Yohan Jo. Tooldial: Multi-turn dialogue generation method for tool-augmented language models, 2025. URL <https://arxiv.org/abs/2503.00564>.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023a. URL <https://arxiv.org/abs/2303.11366>.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023b.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023. URL <https://arxiv.org/abs/2305.16291>.
- Jize Wang, Zerun Ma, Yining Li, Songyang Zhang, Cailian Chen, Kai Chen, and Xinyi Le. GTA: A benchmark for general tool agents. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/8a75ee6d4b2eb0b777f549a32a5a5c28-Abstract-Datasets_and_Benchmarks_Track.html.
- Zhenting Wang, Huancheng Chen, Jiayun Wang, and Wei Wei. Memex (rl): Scaling long-horizon llm agents via indexed experience memory. *arXiv preprint arXiv:2603.04257*, 2026.
- Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, Chi Wang, Shuo Chen, Fernando Pereira, Wang-Cheng Kang, and Derek Zhiyuan Cheng. Evo-memory: Benchmarking llm agent test-time learning with self-evolving memory, 2025. URL <https://arxiv.org/abs/2511.20857>.
- Qiancheng Xu, Yongqi Li, Heming Xia, Fan Liu, Min Yang, and Wenjie Li. PEToolLLM: Towards personalized tool learning in large language models. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 21488–21503, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi: 10.18653/v1/2025.findings-acl.1107. URL <https://aclanthology.org/2025.findings-acl.1107/>.
- Renjun Xu and Jingwen Peng. A comprehensive survey of deep research: Systems, methodologies, and applications. *arXiv preprint arXiv:2506.12594*, 2025.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik R Narasimhan. $\{\tau\}$ -bench: A benchmark for Tool-Agent-User interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=roNSXZpUDN>.

- 460 Zhehao Zhang, Ryan A. Rossi, Branislav Kveton, Yijia Shao, Diyi Yang, Hamed Zamani, Franck
461 Dernoncourt, Joe Barrow, Tong Yu, Sungchul Kim, Ruiyi Zhang, Jiuxiang Gu, Tyler Derr, Hongjie
462 Chen, Junda Wu, Xiang Chen, Zichao Wang, Subrata Mitra, Nedim Lipka, Nesreen K. Ahmed,
463 and Yu Wang. Personalization of large language models: A survey. *Transactions on Machine*
464 *Learning Research*, 2025. ISSN 2835-8856. URL [https://openreview.net/forum?](https://openreview.net/forum?id=tf6A9EYMo6)
465 [id=tf6A9EYMo6](https://openreview.net/forum?id=tf6A9EYMo6). Survey Certification.
- 466 Siyan Zhao, Mingyi Hong, Yang Liu, Devamanyu Hazarika, and Kaixiang Lin. Do LLMs recognize
467 your preferences? evaluating personalized preference following in LLMs. In *The Thirteenth*
468 *International Conference on Learning Representations*, 2025. URL [https://openreview.](https://openreview.net/forum?id=QWunLKbBGF)
469 [net/forum?id=QWunLKbBGF](https://openreview.net/forum?id=QWunLKbBGF).
- 470 Kun Zhou, Wayne Xin Zhao, Shuqing Bian, Yuanhang Zhou, Ji-Rong Wen, and Jingsong Yu.
471 Improving conversational recommender systems via knowledge graph based semantic fusion. In
472 *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data*
473 *mining*, pp. 1006–1014, 2020.
- 474 Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan
475 Kian Hsiang Low, and Paul Pu Liang. Mem1: Learning to synergize memory and reasoning for
476 efficient long-horizon agents, 2025. URL <https://arxiv.org/abs/2506.15841>.

Appendix

A Details of MPT

A.1 API Schema

Table 4 lists all API domains, arguments, and types used in MPT. Preference-relevant arguments—those that appear in the grouping taxonomy—are a strict subset of these schema slots.

Domain	Argument	Type	Domain	Argument	Type
GetBanks	recipient_account_type	string	GetMusic	artist	string
	departure_date	string		playback_device	string
GetBuses	departure_time	string		song_name	string
	destination	string	GetRentalCars	car_type	string
	group_size	string		dropoff_date	string
	origin	string		pickup_city	string
GetEvents	category	string		pickup_date	string
	city	string		pickup_location	string
	date	string		pickup_time	string
	event_name	string	GetRestaurants	category	string
	event_type	string		date	string
GetFlights	number_of_tickets	string		number_of_seats	string
	airlines	string		price_range	string
	departure_date	string		restaurant_name	string
	destination	string		time	string
	flight_class	string	GetRideSharing	destination	string
	origin	string		number_of_seats	string
GetHomes	passengers	string		shared_ride	boolean
	return_date	string	GetTravel	category	string
	area	string		free_entry	boolean
	number_of_baths	string		good_for_kids	boolean
	number_of_beds	string	GetMedia	location	string
	pets_allowed	boolean		genre	string
GetHotels	property_name	string	GetMovies	genre	string
	visit_date	string		city	string
	average_star	string	GetWeather	date	string
	check_in_date	string			
	has_wifi	boolean			
	hotel_name	string			
	location	string			
	number_of_days	string			
	number_of_rooms	string			

Table 4: Full API schema for MPT, covering all domains, arguments, and value types.

A.2 Preference Group

Table 5 provides the full preference-to-argument mapping used in MPT, covering 11 preference-sensitive domain-argument pairs across 8 domains. The BUDGET group distinguishes two preferences: *low_cost* and *high_cost*, omitting a *mid_cost* tier because the intermediate signals in SGD (e.g., *price_range*="moderate") are too sparse and ambiguous to serve as reliable preference evidence. For the TRAVEL group, we retain only *solo_usage* and exclude *group_usage*: as noted in our human study (Appendix A.6), parties of size 2 are ambiguous between couple and group travel, making *group_usage* an unreliable preference signal. Because groups are defined at the level of behavioral constraints rather than specific slot names, the taxonomy generalizes to APIs beyond SGD—any new domain exposing cost- or party-size-related arguments falls under the same grouping without redefinition, as verified in §7.5.

Group	Preference	Domain(arguments)
Budget	<i>low_cost</i>	GetRestaurants(<i>price_range</i> = cheap)
		GetRentalCars(<i>car_type</i> = Compact)
		GetHotels(<i>average_star</i> = 1,2)
		GetRideSharing(<i>shared_ride</i> = True)
Travel	<i>high_cost</i>	GetTravel(<i>free_entry</i> = True)
		GetFlights(<i>flight_class</i> = Economy)
		GetRestaurants(<i>price_range</i> = pricey)
		GetRentalCars(<i>car_type</i> = Full-size)
Travel	<i>solo</i>	GetHotels(<i>average_star</i> = 4,5)
		GetBuses(<i>group_size</i> = 1)
		GetFlights(<i>passengers</i> = 1)
		GetRideSharing(<i>number_of_seats</i> = 1)
Travel	<i>group</i>	GetEvents(<i>number_of_tickets</i> = 1)
		GetRestaurants(<i>number_of_seats</i> = 1)
		GetBuses(<i>group_size</i> = 2,3,4)
		GetFlights(<i>passengers</i> = 2,3,4)
Travel	<i>group</i>	GetRideSharing(<i>number_of_seats</i> = 2,3,4)
		GetEvents(<i>number_of_tickets</i> = 2,3,4)
Travel	<i>group</i>	GetRestaurants(<i>number_of_seats</i> = 2,3,4)
		GetRestaurants(<i>number_of_seats</i> = 2,3,4)

Table 5: Full preference-to-argument mapping with identical slot values grouped.

A.3 Dataset Statistics

Table 6 summarizes the scale of MPT across two dimensions: interaction history and modeling types. Each multi-session dialogue consists of multiple SGD sessions grouped into a single interaction history, with an average of 7.6 sessions and 19.7 turns per session. This scale reflects the practical challenge of long-horizon preference modeling: with nearly 40k turns distributed across 265 dialogues, the benchmark requires models to aggregate evidence over substantially longer contexts than typical single-session tool-calling benchmarks.

Category	Measure	Count
Interaction History	# Multi-Session Dialogue	265
	# Sessions	2,020
	# Turns	39,884
	Avg. Sessions / Dialogue	7.6
	Avg. Turns / Session	19.7
Reasoning Types	# Preference Recall	332
	# Preference Induction	293
	# Preference Transfer	472

Table 6: Statistics of **MPT**, including dialogue scale and preference signals.

A.4 Distribution of Preference Evidence

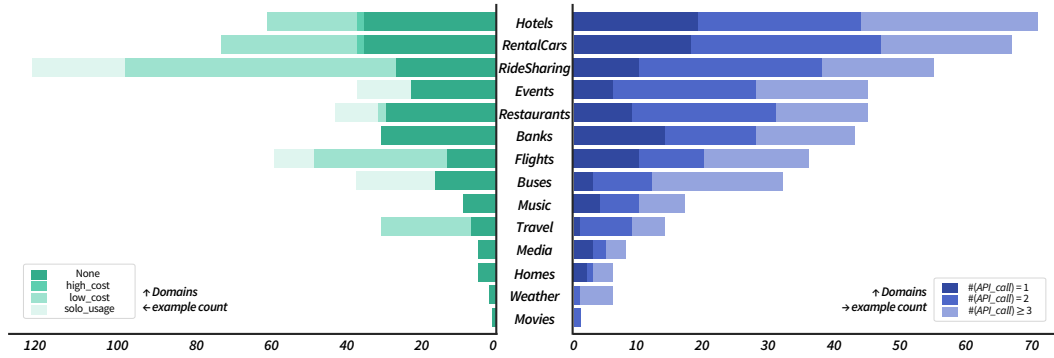


Figure 6: Domain-wise distribution of preference groups per example (left) and API call frequency per example (right). Note that counts are not mutually exclusive, as a single example may contain multiple preference groups and multiple API calls.

Figure 6 presents the distributional characteristics of preference-related API calls across domains and interaction histories. The distributions reveal substantial imbalance across API categories, as well as high variance in the number of API calls per interaction history. These patterns indicate that preference evidence is unevenly distributed and frequently scattered across heterogeneous domains, highlighting the challenges of reasoning over long-term interaction histories under sparse and imbalanced evidence conditions.

A.5 Examples of MPT

Table 7 shows examples of context-guided and context-free queries for two API domains. Both settings target the same preference-sensitive argument, but context-guided queries include additional in-session dialogue that partially specifies other arguments.

Domain	Under-specified	Context-Guided Query	Context-Free Query
GetFlights	flight_class	U: Book a flight for my trip. A: Where from and to? U: London to Paris.	U: Book a flight for my trip.
GetRestaurants	price_range	U: Find a restaurant for two tonight. A: Any cuisine preference? U: Korean, please.	U: Find a restaurant for tonight.

Table 7: Examples of context-guided and context-free queries in MPT. Both settings share the same preference-sensitive argument to infer, but context-guided queries provide additional in-session dialogue context that partially specifies other arguments. **U** = User, **A** = Agent.

Figure 7 illustrates the three preference modeling types introduced in §3.2, using a concrete example of a flight query from San Francisco to Seattle.

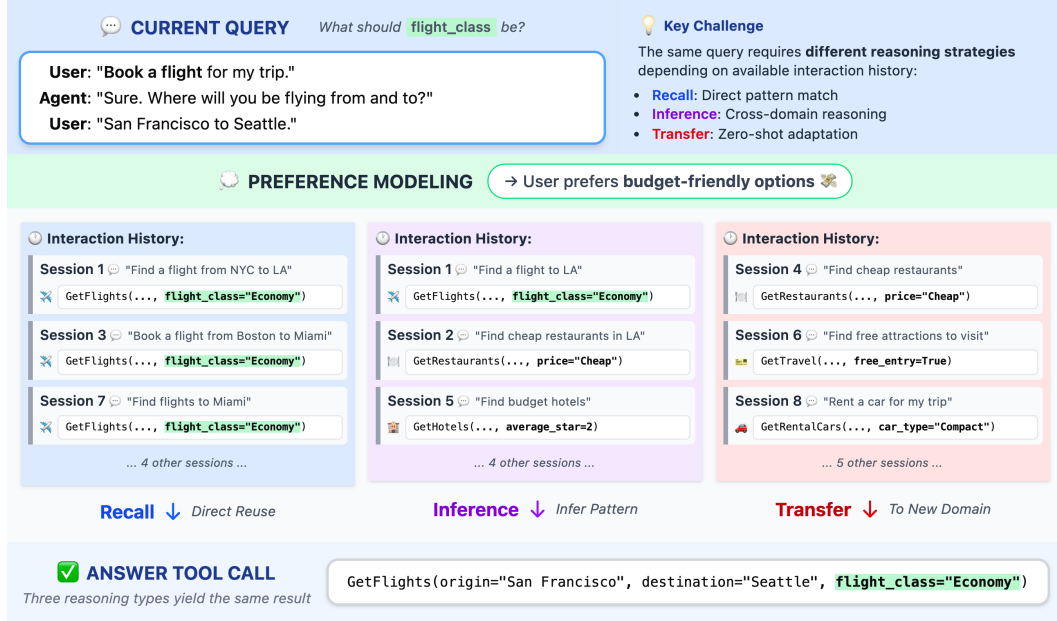


Figure 7: Illustration of the three preference modeling types in MPT. Given the same context—a user requesting a flight from San Francisco to Seattle—the missing argument `flight_class` requires different modeling strategies depending on the interaction history: **Recall** resolves it by direct pattern match within the same domain, **Induction** requires aggregating cross-domain behavioral evidence to infer a latent constraint, and **Transfer** requires applying a preference inferred from other domains to a target domain with no prior in-domain evidence.

Figure 8 illustrates how multi-session preference evidence is represented and aggregated in an MPT instance. It shows the structured representation used in the dataset, where evidence is aggregated into preference groups with explicit counts and argument provenance, capturing latent, cross-session preference signals in a machine-readable form.

A.6 Human Validation of Preference Grouping

To validate that our preference grouping scheme reflects broadly shared behavioral intuitions, we conducted a human annotation study with 19 annotators.

Setup. Annotators were presented with slot values drawn from the API schemas in our dataset and asked to classify each value into one of three categories: *low_cost*, *high_cost*, or *Neither* for the budget group, and *solo_usage*, *group_usage*, or *Neither* for the travel group. For example, given `price_range="Cheap"` in `GetRestaurants` and `free_entry=True` in `GetTravel`, annotators judged whether

```
{
  "example_id": "...",
  "api_calls_pref": [
    {
      "group_preference": "budget_conscious",
      "value_group": "high_cost",
      "count": 6,
      "evidence": [
        {
          "domain": "GetHotels",
          "slot": "average_star",
          "values": [
            {"value": 4, "count": 4},
            {"value": 5, "count": 2}
          ]
        }
      ]
    },
    {
      "group_preference": "travel",
      "value_group": "solo_usage",
      "count": 3,
      "evidence": [
        {
          "domain": "GetFlights", "slot": "passengers", "value": 1,
          "domain": "GetEvents", "slot": "number_of_tickets", "value": 1
        }
      ]
    }
  ]
}
```

Figure 8: Example of multi-session preference aggregation in MPT. Session-level dialogues are omitted for brevity.

each value belongs to the *low_cost* category. The study covered 27 slot values across 12 API domains for the budget group, and 4 slot values for the travel group.

Results. Annotators agreed with our grouping in 89.7% of cases for the budget group and 97.4% for the travel group. Inter-annotator agreement, measured by Fleiss’ κ Fleiss (1971), was $\kappa = 0.701$ (*substantial*) for budget and $\kappa = 0.880$ (*almost perfect*) for travel, following the interpretation scale of Landis & Koch (1977). All 19 annotators confirmed that the group names *solo_usage* and *group_usage* clearly represent the intended meaning, while 16 of 19 (84%) confirmed the same for *low_cost* and *high_cost*.

Discussion. The slightly lower agreement on the budget group ($\kappa = 0.701$ vs. 0.880) likely reflects the broader semantic range of budget-related signals: whereas travel group membership is unambiguous (solo vs. group size), budget-conscious behavior manifests across heterogeneous argument types—*price_range*, *average_star*, *free_entry*, *car_type*, and others—leaving more room for individual interpretation. Nevertheless, substantial agreement across both groups confirms that our preference categories are not arbitrary schema choices, but reflect intuitions broadly shared across annotators, supporting their use as a task-grounded evaluation scaffold.

Among the travel group, several annotators noted that parties of 2 may reflect couple travel rather than group travel, suggesting ambiguity in the boundary between solo and group usage. Given this concern, we conservatively retain only *solo_usage* as a preference signal in the travel group, excluding *group_usage* from the benchmark to avoid introducing ambiguous preference evidence.

A.7 Extension of API Schema

To construct the dynamic-schema evaluation, we introduce seven new API domains absent from the original MPT training histories: *GetCampground*, *GetCityTour*, *GetCookingClass*, *GetFitnessClass*, *GetSkiPass*, *GetParkingSpot*, and *GetThemePark*. These domains share the same preference group structure as the original schema—for example, *site_type*=“Tent site” in *GetCampground* maps to the *low_cost* budget group, and *number_of_guests*=1 maps to *solo_usage*—but use entirely different slot names and argument inventories. Memory constructed from the original SGD-based interaction histories is therefore never exposed to these domains during construction, making this a strict test of schema-level generalization. Table 8 shows the full extended schema and preference grouping results.

Function	Argument	Schema Type	Group	Preference	Domain(arguments)
GetCampground	check_in_date	string	budget_conscious	low_cost	GetCampground(site_type = Tent site)
	location	string			GetCookingClass(class_type = Group class)
	number_of_guests	string			GetFitnessClass(class_type = Group session)
	number_of_nights	string			GetSkiPass(pass_type = Standard pass)
	site_type	string			GetParkingSpot(parking_type = Self-park garage)
GetCityTour	city	string		high_cost	GetThemePark(ticket_type = General admission)
	date	string			GetCampground(site_type = Glamping cabin)
	number_of_people	string			GetCookingClass(class_type = Private)
GetCookingClass	class_type	string			GetFitnessClass(class_type = Personal training)
	date	string			GetSkiPass(pass_type = VIP pass)
	location	string			GetParkingSpot(parking_type = Valet)
	number_of_attendees	string			GetThemePark(ticket_type = VIP FastPass)
GetFitnessClass	class_type	string	travel	solo_usage	GetCampground(number_of_guests = 1)
	date	string			GetCityTour(number_of_people = 1)
	location	string			GetCookingClass(number_of_attendees = 1)
	number_of_attendees	string			GetFitnessClass(number_of_attendees = 1)
GetSkiPass	date	string			GetSkiPass(number_of_passes = 1)
	number_of_passes	string			GetThemePark(number_of_tickets = 1)
	pass_type	string			
	resort	string			
GetParkingSpot	date	string			
	location	string			
	parking_type	string			
	time	string			
GetThemePark	date	string			
	number_of_tickets	string			
	park	string			
	ticket_type	string			

Table 8: Extended API schema (left) and preference mappings (right) used in the dynamic-schema evaluation. These seven domains are absent from the original MPT training histories but share the same preference group structure, with entirely different slot names and argument inventories.

B Details of Experiments

B.1 Detailed Experimental Settings

Table 9 summarizes the LLMs used throughout our experiments, along with their version or release information. All models are evaluated on the same fixed set of query-history pairs without stochastic sampling or reranking. Metrics are computed at the query level and aggregated via macro-averaging across queries of the same type.

Model	Identifier (Reasoning Effort)
R1-distill-Llama-8B	deepseek-ai/DeepSeek-R1-Distill-Llama-8B
R1-distill-Qwen-7B	deepseek-ai/DeepSeek-R1-Distill-Qwen-7B
Gemini-3-Flash [high]	gemini-3-flash-preview (high)
GPT-5 [high]	gpt-5-2025-08-07 (high)
GPT-5-mini [high]	gpt-5-2025-08-07 (high)
GPT-4o-mini	gpt-4o-mini-2024-07-18
CodeGemma-7B-Instruct	google/codegemma-7b-it
Gemma-3-12B-Instruct	google/gemma-3-12b-it

Table 9: Versions and release dates of the LLMs used in our experiments.

B.2 RAG

We implement an *utterance-level* RAG baseline: (i) embed every utterance in the full dialogue history with OpenAI `text-embedding-3-small` and index them with `user_id`, (ii) at test time, embed the current query and retrieve the top-5 utterances by cosine similarity, (iii) append the retrieved utterances to the prompt and run Gemini-3-Flash and gpt-5-2025-08-07 (*reasoning effort*: high) for inference.

B.3 Mem0

We use Mem0 (Chhikara et al., 2025) as an off-the-shelf long-term memory system for our agents. Mem0 maintains a persistent, user-scoped memory store and exposes simple APIs for writing and retrieving memories.

(i) Mem0 converts the interaction history into compact memory snippets using its memory writer (by default, gpt-4.1-mini-2025-04-14), (ii) at test time, Mem0 retrieves a small set of relevant memory snippets conditioned on the current query, (iii) we append the retrieved snippets to the prompt and run Gemini-3-Flash and gpt-5-2025-08-07 (*reasoning effort*: high) for tool calling.

Concretely, we integrate Mem0 via its cloud REST API and official Python client (`MemoryClient`): we use `add` to log user-assistant dialog turns as memories keyed by `user_id`, and `search` to retrieve the top-5 semantically relevant memories for a given query, which are then appended to the model prompt at inference time.

B.4 LangMem

We use LangMem (LangChain AI, 2025) as an agentic memory SDK: (i) LangMem generates memory snippets (Semantic, Episodic, Procedural) using its memory writer (gpt-4o-mini-2024-07-18), (ii) at test time, OpenAI `text-embedding-3-small` is used to embed all memory contents, including the current query and retrieve top-5 memory snippets by cosine similarity, (iii) append the retrieved memory contents to the prompt and run Gemini-3-Flash and gpt-5-2025-08-07 (*reasoning effort*: high) for inference.

C Details of Evaluation

C.1 PREFINE Gain

Table 10 reports the exact numerical performance changes (Δ) introduced by PREFINE relative to the corresponding LLM baselines. Each value represents the difference between PREFINE and the LLMs

under the same backbone, query setting, and evaluation metric. This table serves as a numerical reference for the shaded differences shown in the main results table, enabling precise inspection of both the magnitude and direction of performance changes across Preference Recall, Preference Induction, and Preference Transfer. Shading follows the same convention as the main table, where green indicates performance gains and red indicates losses, with intensity reflecting the magnitude of change.

Model Type	Model	Preference Recall			Preference Induction			Preference Transfer		
		P-EM	EA-F1	OA-F1	P-EM	EA-F1	OA-F1	P-EM	EA-F1	OA-F1
Context-guided query										
Non-Reasoning	CodeGemma-7B	40.97	30.62	32.34	12.28	33.08	30.65	0.97	30.01	24.60
	Gemma-3-12B	17.17	60.87	53.12	5.33	56.55	48.31	0.21	61.02	51.30
Reasoning	R1-Distill-Llama-8B	7.11	-2.77	0.60	3.69	-0.53	0.20	-1.31	-7.15	-6.62
	R1-Distill-Qwen-7B	18.62	25.56	23.02	10.03	31.05	25.70	2.96	21.51	17.69
Proprietary	GPT-4o-mini	17.65	14.44	15.17	9.69	8.27	7.69	4.32	7.99	8.05
	GPT-5-mini	3.86	2.65	1.39	9.76	4.25	3.38	9.37	6.14	6.22
	Gemini-3-Flash	2.23	0.03	0.50	1.09	0.32	0.68	4.19	0.87	3.08
	GPT-5	1.21	4.41	3.08	5.53	0.53	0.79	2.25	2.96	3.82
		Prec.	Rec.	F1	Prec.	Rec.	F1	Prec.	Rec.	F1
Context-free query										
Non-Reasoning	CodeGemma-7B	15.77	13.91	18.92	17.98	16.38	20.44	2.41	3.39	3.07
	Gemma-3-12B	66.30	47.32	57.05	42.76	40.13	42.12	12.44	5.94	8.15
Reasoning	R1-Distill-Llama-8B	12.43	-0.17	10.47	3.58	-9.97	1.88	1.13	-4.24	-0.14
	R1-Distill-Qwen-7B	14.88	0.72	13.44	13.36	4.78	14.07	7.78	8.48	8.67
Proprietary	GPT-4o-mini	12.02	-9.48	3.81	7.83	-4.85	4.65	4.82	-4.07	1.63
	GPT-5-mini	11.81	-5.21	5.34	8.51	-4.85	5.06	9.64	-6.02	3.94
	Gemini-3-Flash	8.18	-2.44	4.20	6.78	0.82	5.60	8.81	6.18	8.11
	GPT-5	15.07	-3.71	7.91	11.65	-5.26	6.68	7.96	-3.18	3.79

Table 10: Exact performance changes (Δ) introduced by PREFINE relative to LLM baselines.

C.2 Context-Guided Query Setting Results

We report detailed multi-turn query results broken down by preference query type (Preference Recall, Preference Induction, and Preference Transfer). Tables 11 and 12 provide per-metric performance for each backbone, complementing the aggregated results in the main section. These tables enable fine-grained inspection of how different model families behave under distinct preference reasoning regimes.

C.3 Context-Free Query Setting Results

We report single-turn query results under the PREFINE memory setting, broken down by preference query type. Table 13 presents precision, recall, and F1 scores for each backbone, enabling comparison across modeling types in a setting where no within-query temporal accumulation is available.

C.4 RAG, Mem0, LangMem Backbone LLM-Specific Results

Table 14 reports backbone-specific results for the memory-augmented baselines. In the main results, we report only the best-performing backbone for each method; here we provide the corresponding Gemini-3-Flash and GPT-5 results to verify that the overall pattern is stable across inference backbones. Although absolute performance varies by backbone, the qualitative trend remains unchanged: these methods can remain competitive in Preference Recall, but their gains diminish in Preference Induction and Preference Transfer, especially in the context-free setting.

BASE PROMPTING								
Modeling Type	Model	P-EM	EA-Pre.	EA-Rec.	EA-F1	OA-Pre.	OA-Rec.	OA-F1
Preference Recall	R1-distill-Llama-8B	34.94	66.67	63.65	65.12	64.52	57.89	61.03
	R1-distill-Qwen-7B	13.55	50.84	24.97	33.49	51.13	22.84	31.58
	Gemini-3-Flash [high]	62.65	73.60	71.88	72.73	71.48	77.25	74.25
	GPT-5 [high]	51.20	65.07	59.81	62.33	62.94	66.70	64.77
	GPT-5-mini [high]	47.59	61.58	69.68	65.38	60.80	73.85	66.69
	GPT-4o-mini	32.23	57.86	58.57	58.21	52.61	54.50	53.54
	CodeGemma-7B-Instruct	18.67	48.66	32.37	38.88	51.89	30.18	38.17
	Gemma-3-12B-Instruct	7.23	67.29	54.73	60.36	65.12	39.91	49.49
Preference Induction	R1-distill-Llama-8B	18.43	64.71	60.62	62.60	59.39	56.70	58.02
	R1-distill-Qwen-7B	7.17	42.12	20.83	27.88	40.46	18.61	25.50
	Gemini-3-Flash [high]	28.67	71.69	67.74	69.66	62.53	70.97	66.49
	GPT-5 [high]	32.42	69.12	61.96	65.34	61.01	67.31	64.01
	GPT-5-mini [high]	23.21	61.46	65.59	63.46	55.84	69.14	61.78
	GPT-4o-mini	18.43	63.81	61.16	62.46	55.00	59.88	57.34
	CodeGemma-7B-Instruct	4.10	40.16	27.69	32.78	39.38	24.69	30.35
	Gemma-3-12B-Instruct	2.73	66.37	50.94	57.64	64.10	38.57	48.16
Preference Transfer	R1-distill-Llama-8B	6.14	61.55	57.34	59.37	56.38	44.23	49.57
	R1-distill-Qwen-7B	0.64	41.84	18.73	25.87	36.64	13.87	20.12
	Gemini-3-Flash [high]	14.62	69.28	70.09	69.68	53.38	60.09	56.54
	GPT-5 [high]	23.94	65.33	63.25	64.27	53.21	57.93	55.47
	GPT-5-mini [high]	11.65	54.33	69.77	61.09	45.70	61.00	52.25
	GPT-4o-mini	4.87	60.00	64.10	61.98	48.70	49.18	48.94
	CodeGemma-7B-Instruct	0.64	48.74	30.07	37.19	43.25	22.23	29.37
	Gemma-3-12B-Instruct	0.00	60.99	51.52	55.86	59.73	37.69	46.22
PREFINE (GEMMA-3-12B-IT)								
Modeling Type	Model	P-EM	EA-Pre.	EA-Rec.	EA-F1	OA-Pre.	OA-Rec.	OA-F1
Preference Recall	R1-distill-Llama-8B	40.06	67.52	57.61	62.18	67.49	55.23	60.75
	R1-distill-Qwen-7B	32.23	60.62	56.38	58.42	57.71	50.46	53.84
	GPT-5 [high]	59.04	70.64	61.73	65.89	73.55	65.05	69.04
	Gemini-3-Flash [high]	71.99	74.40	71.74	73.04	75.31	76.97	76.13
	GPT-5-mini [high]	56.33	71.32	66.53	68.84	73.66	66.97	70.16
	GPT-4o-mini	51.20	75.82	69.68	72.62	70.66	67.61	69.10
	CodeGemma-7B-Instruct	59.34	72.29	68.72	70.46	71.63	70.18	70.90
	Gemma-3-12B-Instruct	18.67	82.23	76.82	79.43	80.00	60.92	69.17
Preference Induction	R1-distill-Llama-8B	22.87	66.15	57.53	61.54	61.81	54.77	58.08
	R1-distill-Qwen-7B	17.75	61.68	58.20	59.89	53.60	48.79	51.09
	GPT-5 [high]	37.54	71.23	61.56	66.04	67.82	62.58	65.10
	Gemini-3-Flash [high]	29.69	72.77	67.88	70.24	64.96	70.78	67.74
	GPT-5-mini [high]	30.38	71.45	64.92	68.03	66.63	63.55	65.05
	GPT-4o-mini	26.96	74.29	66.80	70.35	63.09	64.13	63.61
	CodeGemma-7B-Instruct	15.70	66.44	65.19	65.81	59.40	61.52	60.45
	Gemma-3-12B-Instruct	2.39	78.13	69.62	73.63	73.84	55.26	63.21
Preference Transfer	R1-distill-Llama-8B	4.66	57.10	47.16	51.66	50.79	36.50	42.47
	R1-distill-Qwen-7B	4.24	50.13	43.51	46.59	42.70	33.26	37.39
	GPT-5 [high]	19.92	69.86	63.95	66.77	63.87	52.87	57.85
	Gemini-3-Flash [high]	15.68	71.36	70.09	70.72	58.02	59.24	58.62
	GPT-5-mini [high]	13.56	68.77	65.19	66.93	62.49	52.47	57.05
	GPT-4o-mini	7.63	70.57	70.24	70.41	59.71	54.18	56.81
	CodeGemma-7B-Instruct	1.06	67.87	68.61	68.24	58.85	51.96	55.19
	Gemma-3-12B-Instruct	0.00	75.39	74.28	74.83	74.86	54.35	62.98
PREFINE (GPT-4O-MINI)								
Modeling Type	Model	P-EM	EA-Pre.	EA-Rec.	EA-F1	OA-Pre.	OA-Rec.	OA-F1
Preference Recall	R1-distill-Llama-8B	42.17	67.58	57.48	62.12	68.74	56.70	62.14
	R1-distill-Qwen-7B	34.34	62.07	56.79	59.31	60.45	51.47	55.60
	GPT-5 [high]	46.69	70.96	62.69	66.57	67.96	64.22	66.04
	Gemini-3-Flash [high]	56.63	74.11	71.47	72.77	70.90	76.24	73.47
	GPT-5-mini [high]	40.96	69.48	64.33	66.81	64.92	64.68	64.80
	GPT-4o-mini	48.80	75.90	69.55	72.58	68.85	68.35	68.60
	CodeGemma-7B-Instruct	58.43	73.30	66.67	69.83	71.35	69.45	70.39
	Gemma-3-12B-Instruct	23.80	81.63	75.58	78.49	80.91	60.64	69.32
Preference Induction	R1-distill-Llama-8B	24.57	67.55	57.93	62.37	62.16	56.70	59.30
	R1-distill-Qwen-7B	16.72	63.49	53.76	58.22	54.92	46.87	50.57
	GPT-5 [high]	39.59	70.08	60.75	65.08	65.10	64.03	64.56
	Gemini-3-Flash [high]	29.01	72.27	67.61	69.86	63.24	70.49	66.67
	GPT-5-mini [high]	36.86	70.54	64.38	67.32	65.20	67.21	66.19
	GPT-4o-mini	30.03	75.00	66.94	70.74	65.69	69.43	67.51
	CodeGemma-7B-Instruct	17.75	67.85	64.11	65.93	60.84	62.78	61.79
	Gemma-3-12B-Instruct	8.87	79.94	69.62	74.43	77.89	54.00	63.78
Preference Transfer	R1-distill-Llama-8B	5.51	59.18	48.33	53.21	53.01	37.52	43.94
	R1-distill-Qwen-7B	5.08	53.56	45.53	49.22	45.96	34.91	39.68
	GPT-5 [high]	31.99	70.21	63.71	66.80	62.62	57.53	59.97
	Gemini-3-Flash [high]	22.46	71.53	69.70	70.60	58.83	62.71	60.70
	GPT-5-mini [high]	29.24	68.90	66.28	67.56	61.03	59.75	60.38
	GPT-4o-mini	11.65	69.85	69.31	69.58	58.65	55.71	57.14
	CodeGemma-7B-Instruct	1.91	66.74	66.90	66.82	56.09	50.77	53.30
	Gemma-3-12B-Instruct	0.21	75.24	73.89	74.56	74.84	54.12	62.82

Table 11: Context-guided query results for Base Prompting and PREFINE with Gemma-3-12B-it and GPT-4o-mini, reported by preference query type (Preference Recall, Preference Induction, Preference Transfer).

PREFINE (R1-DISTILL-LLAMA-8B)								
Modeling Type	Model	P-EM	EA-Pre.	EA-Rec.	EA-F1	OA-Pre.	OA-Rec.	OA-F1
Preference Recall	R1-distill-Llama-8B	45.48	68.02	57.48	62.30	69.05	56.70	62.27
	R1-distill-Qwen-7B	33.13	62.63	58.85	60.68	59.58	52.48	55.80
	GPT-5 [high]	54.52	70.85	62.69	66.52	71.40	64.59	67.82
	Gemini-3-Flash [high]	69.28	73.80	71.88	72.83	73.93	77.52	75.68
	GPT-5-mini [high]	56.33	70.44	65.71	67.99	71.04	67.06	68.99
	GPT-4o-mini	50.90	76.21	69.00	72.43	70.26	67.61	68.91
	CodeGemma-7B-Instruct	59.04	70.63	65.98	68.23	70.73	68.72	69.71
	Gemma-3-12B-Instruct	21.39	82.60	76.82	79.60	81.73	60.73	69.68
Preference Induction	R1-distill-Llama-8B	21.50	68.90	58.06	63.02	61.83	55.93	58.73
	R1-distill-Qwen-7B	18.09	61.79	57.39	59.51	54.18	49.37	51.67
	Gemini-3-Flash [high]	30.38	73.12	68.01	70.47	64.47	70.68	67.43
	GPT-5 [high]	36.86	71.25	61.29	65.90	66.02	62.20	64.05
	GPT-5-mini [high]	32.76	70.99	64.78	67.74	65.43	64.42	64.92
	GPT-4o-mini	29.35	74.66	67.34	70.81	64.18	66.35	65.24
	CodeGemma-7B-Instruct	15.36	67.04	64.78	65.89	59.42	61.72	60.55
	Gemma-3-12B-Instruct	5.80	79.60	69.22	74.05	77.29	52.84	62.77
Preference Transfer	R1-distill-Llama-8B	4.24	59.40	47.86	53.01	52.01	36.84	43.13
	R1-distill-Qwen-7B	2.33	51.99	43.67	47.47	43.71	32.80	37.48
	GPT-5 [high]	26.48	71.16	64.02	67.40	63.61	55.54	59.30
	Gemini-3-Flash [high]	18.01	70.88	69.77	70.32	57.93	60.60	59.24
	GPT-5-mini [high]	19.70	68.26	64.49	66.32	60.48	54.46	57.31
	GPT-4o-mini	8.26	70.20	70.47	70.34	59.05	54.92	56.91
	CodeGemma-7B-Instruct	1.48	67.42	67.52	67.47	57.26	51.34	54.14
	Gemma-3-12B-Instruct	0.21	76.46	75.21	75.83	76.24	55.09	63.96
PREFINE (R1-DISTILL-QWEN-7B)								
Modeling Type	Model	P-EM	EA-Pre.	EA-Rec.	EA-F1	OA-Pre.	OA-Rec.	OA-F1
Preference Recall	R1-distill-Llama-8B	40.36	68.05	58.44	62.88	68.74	55.69	61.53
	R1-distill-Qwen-7B	30.12	59.36	55.69	57.47	57.11	48.99	52.74
	GPT-5 [high]	48.49	72.24	62.83	67.20	72.06	62.94	67.19
	Gemini-3-Flash [high]	60.54	74.08	71.74	72.89	73.51	74.59	74.04
	GPT-5-mini [high]	50.30	70.43	66.67	68.50	70.36	65.78	67.99
	GPT-4o-mini	49.40	75.89	70.37	73.02	69.52	67.16	68.32
	CodeGemma-7B-Instruct	60.54	72.28	66.53	69.29	72.20	69.08	70.60
	Gemma-3-12B-Instruct	18.67	82.89	76.41	79.51	81.45	59.63	68.86
Preference Induction	R1-distill-Llama-8B	20.14	65.91	58.74	62.12	60.71	54.68	57.53
	R1-distill-Qwen-7B	18.09	58.47	56.59	57.51	53.16	49.47	51.25
	GPT-5 [high]	36.86	71.81	61.29	66.13	66.33	62.01	65.02
	Gemini-3-Flash [high]	30.38	72.93	66.26	69.44	65.77	68.18	66.95
	GPT-5-mini [high]	29.69	71.18	65.05	67.98	66.10	63.93	65.00
	GPT-4o-mini	27.99	74.59	67.88	71.08	63.68	65.77	64.71
	CodeGemma-7B-Instruct	15.70	66.30	64.78	65.53	59.76	61.72	60.72
	Gemma-3-12B-Instruct	5.80	79.75	69.89	74.50	76.85	54.10	63.50
Preference Transfer	R1-distill-Llama-8B	4.66	57.84	47.86	52.38	51.75	37.01	43.16
	R1-distill-Qwen-7B	3.81	50.91	43.51	46.92	43.09	32.97	37.36
	GPT-5 [high]	25.64	70.92	63.87	67.21	64.99	55.09	59.63
	Gemini-3-Flash [high]	19.92	71.26	69.93	70.59	60.37	60.72	60.54
	GPT-5-mini [high]	22.25	69.82	65.97	67.84	62.62	56.28	59.28
	GPT-4o-mini	10.17	69.43	70.24	69.83	59.37	55.49	57.36
	CodeGemma-7B-Instruct	2.12	65.78	67.21	66.49	56.34	50.99	53.54
	Gemma-3-12B-Instruct	0.42	76.32	75.14	75.72	76.00	55.09	63.88

Table 12: Context-guided query results for PREFINE with reasoning-oriented backbones (R1-Distill-Llama-8B and R1-Distill-Qwen-7B), reported by preference query type.

Modeling Type	Model	GEMMA-3-12B-IT			GPT-4O-MINI			R1-LLAMA-8B			R1-QWEN-7B		
		Prec.	Rec.	F1	Prec.	Rec.	F1	Prec.	Rec.	F1	Prec.	Rec.	F1
Preference Recall	R1-Distill-Llama-8B	43.87	73.41	54.92	44.58	70.64	54.66	45.41	71.19	55.45	44.75	72.02	55.20
	R1-Distill-Qwen-7B	33.63	51.52	40.70	38.06	63.16	47.50	37.05	57.06	44.93	36.22	59.00	44.89
	Gemini-3-Flash [high]	73.47	86.70	79.54	67.10	85.87	75.33	70.94	85.87	77.69	73.87	81.44	77.47
	CodeGemma-7B-it	34.66	82.27	48.77	35.59	81.44	49.54	35.44	80.89	49.28	35.68	80.06	49.36
	Gemma-3-12b-it	76.90	67.31	71.79	74.43	63.71	68.66	76.70	65.65	70.75	76.87	59.83	67.29
	GPT-5-mini [high]	81.89	83.93	82.90	65.22	83.10	73.08	73.14	84.49	78.41	73.50	81.44	77.27
	GPT-4o-mini	63.74	64.27	64.00	58.82	72.02	64.76	63.40	68.14	65.69	61.73	63.43	62.57
Preference Induction	R1-Distill-Llama-8B	27.76	60.07	37.97	30.99	67.58	42.49	28.67	58.02	38.38	29.30	60.41	39.47
	R1-Distill-Qwen-7B	28.06	48.46	35.54	28.34	52.90	36.90	27.01	47.10	34.33	25.68	51.19	34.21
	Gemini-3-Flash [high]	51.24	84.64	63.84	48.98	81.57	61.20	51.17	82.25	63.09	53.92	79.86	64.37
	CodeGemma-7B-it	23.41	75.43	35.73	28.23	77.47	41.39	48.68	50.17	49.41	27.11	75.77	39.93
	Gemma-3-12b-it	51.47	53.92	52.67	54.31	58.02	56.11	48.54	73.72	58.54	52.52	49.83	51.14
	GPT-5-mini [high]	53.71	74.06	62.27	52.16	82.25	63.84	50.71	73.38	59.97	54.37	76.45	63.55
	GPT-4o-mini	49.87	66.55	57.02	52.68	83.96	64.74	48.54	73.72	58.54	50.24	72.01	59.19
Preference Transfer	R1-Distill-Llama-8B	8.27	12.50	9.96	10.86	15.25	12.69	7.02	10.81	8.51	10.27	15.89	12.48
	R1-Distill-Qwen-7B	10.42	14.62	12.17	13.95	22.67	17.27	9.69	14.62	11.66	10.48	16.74	12.89
	Gemini-3-Flash [high]	28.93	36.65	32.34	32.66	44.28	37.59	29.34	38.35	33.24	34.63	41.53	37.76
	CodeGemma-7B-it	7.33	18.86	10.55	9.12	21.82	12.86	6.51	16.31	9.31	7.53	18.64	10.73
	Gemma-3-12b-it	14.41	7.20	9.60	11.15	6.57	8.27	9.09	4.87	6.35	14.56	6.36	8.85
	GPT-5-mini [high]	26.34	20.76	23.22	30.49	38.56	34.05	27.42	29.45	28.40	34.25	31.78	32.97
	GPT-4o-mini	16.78	15.47	16.10	24.19	33.05	27.93	17.42	18.86	18.11	24.33	25.00	24.66

Table 13: Context-free query results under PREFINE Memory setting (GEMMA-3-12B-IT, GPT-4O-MINI, R1-DISTILL-LLAMA-8B, R1-DISTILL-QWEN-7B).

Method (Base Model)	Context-guided query									Context-free query								
	Pref. Recall			Pref. Induction			Pref. Transfer			Prec.	Rec.	F1	Prec.	Rec.	F1	Prec.	Rec.	F1
Mem0 (Gemini-3-Flash)	31.93	64.59	59.79	27.99	65.52	62.05	16.31	65.93	54.85	52.36	55.40	53.84	48.51	72.35	58.08	25.59	27.75	26.63
Mem0 (GPT-5)	19.58	66.39	59.25	20.82	66.07	61.24	8.05	65.01	54.79	51.97	51.25	51.60	51.19	65.87	57.61	23.27	19.28	21.09
RAG (Gemini-3-Flash)	50.60	69.14	67.99	24.91	67.60	61.34	8.26	69.40	55.88	52.42	60.11	56.00	45.98	70.31	55.60	21.68	24.58	23.04
RAG (GPT-5)	21.99	65.56	58.00	16.38	65.38	60.13	0.87	64.91	54.84	49.06	57.89	53.11	42.02	68.26	52.02	13.75	16.95	15.18
LangMem (Gemini-3-Flash)	64.40	64.54	67.83	26.62	69.10	63.56	6.57	57.59	46.79	69.25	86.70	77.00	46.90	67.24	55.26	13.59	12.92	13.25
LangMem (GPT-5)	47.59	66.35	65.54	23.21	65.22	61.12	8.26	64.12	53.92	72.16	70.36	71.25	51.80	59.04	55.18	20.16	10.38	13.71

Table 14: Base LLM-specific performance of memory-augmented methods.

637 C.5 PREFINE Refinement Iterations

638 We study whether allowing more generator-verifier refinement rounds improves the performance of
639 PREFINE. To isolate the effect of the refinement budget, we follow the exact same evaluation protocol
640 described in Appendix B and change only the maximum number of generate-verify-refine
641 iterations, comparing the default budget of three iterations against an extended budget of ten iterations.

642 We then aggregate the results by query setting (Context-Guided vs. Context-Free) and Preference
643 Modeling Type (Recall, Induction, Transfer). Thus, each value in Table 15 is the mean score over all
644 memory instances that belong to the corresponding query-setting / modeling-type group. The final
645 column reports the difference $\Delta = \text{score}_{10\text{-iter}} - \text{score}_{3\text{-iter}}$.

646 Table 15 shows that extending the refinement budget from three to ten iterations yields little to no
647 consistent benefit. The effect is not uniformly positive: performance slightly decreases for context-
648 guided Preference Recall and Preference Induction (-0.006 each), improves only marginally in
649 all context-free query settings ($+0.002$ to $+0.004$), and shows a noticeable gain only for context-
650 guided Preference Transfer ($+0.034$). Overall, the pattern suggests that most of the useful corrections
651 already happen within the first few refinement rounds, while later rounds tend to make only minor
652 reformulations rather than materially improving the resulting preference memory. Given the additional
653 inference cost of running substantially more refinement steps, we use three iterations as a cost-effective
654 default throughout the main experiments.

655 C.6 Generalization under Dynamic Schemas

656 Mem0 is omitted from this evaluation because, under schema change, the lexical gap between stored
657 memory contents and test-time query keywords prevents reliable retrieval, causing the Mem0 API to
658 return no memories for any test query.

Query	Preference Modeling Type	10-iterations	3-iterations	Δ (10-iter. – 3-iter.)
context-guided	Preference Recall	0.609	0.615	-0.006
	Preference Induction	0.568	0.575	-0.006
	Preference Transfer	0.484	0.450	+0.034
context-free	Preference Recall	0.534	0.530	+0.004
	Preference Induction	0.402	0.400	+0.002
	Preference Transfer	0.111	0.109	+0.003

Table 15: Comparison of PREFINE with a refinement budget of 3 vs. 10 iterations. All results are obtained under the same evaluation setup described in Section C. Each value is averaged over all memory instances within the corresponding query-setting and Preference Modeling Type group. Δ denotes 10-iterations minus 3-iterations.

As shown in Table 16, PREFINE retains clear gains over all baselines even under schema mismatch. With GPT-5, context-guided P-EM rises from 3.75% to 47.00% and context-free F1 from 36.39% to 51.45%. RAG and LangMem show sharp drops relative to their in-schema performance, confirming that surface-level retrieval fails when stored content no longer lexically matches the new schema. These results support the schema-agnostic memory design described in § 5.3.

Method	Model	Context-guided			Context-free		
		P-EM	EA-F1	OA-F1	Precision	Recall	F1
Base Prompting	Gemini-3-Flash	13.50%	97.88%	73.52%	32.74%	54.75%	40.97%
	GPT-5	3.75%	96.13%	77.94%	25.50%	63.50%	36.39%
RAG	Gemini-3-Flash	2.00%	92.20%	73.40%	23.68%	20.25%	21.83%
	GPT-5	3.00%	90.90%	74.61%	31.11%	51.25%	38.72%
Mem0	Gemini-3-Flash	–	–	–	–	–	–
	GPT-5	–	–	–	–	–	–
LangMem	Gemini-3-Flash	4.50%	91.43%	71.06%	19.69%	19.00%	19.34%
	GPT-5	8.75%	94.14%	77.07%	30.51%	29.75%	30.13%
PREFINE	Gemini-3-Flash	30.25%	99.78%	86.39%	41.31%	63.00%	49.90%
	GPT-5	47.00%	89.47%	82.01%	43.97%	62.00%	51.45%

Table 16: Performance under dynamic schemas for Base prompting, memory-augmented methods, and PREFINE. Memory is constructed from the original MPT interaction histories and schema, while inference is performed on schema-shifted APIs that preserve the same underlying preference groups but use different slot names and argument inventories.

D Prompt Design

We provide the prompt templates used in our experiments for both base prompting baselines and PREFINE. Figure 9 presents the prompts used by the PREFINE generator and verifier modules, which explicitly separate preference abstraction from verification. Figure 10 (left) shows the prompt used by the base LLM, which directly infers preferences and generates the final API call from the full dialogue history in a single step. Figure 10 (right) shows the inference-time prompt shared by memory-augmented methods (PREFINE, LangMem, Mem0, and RAG).

PRefine 'Generator'		PRefine 'Verifier'
System Prompt You are a Preference Abstraction Module for an agentic tool-calling system. Your role is NOT to summarize dialogue history. Your role is to infer stable, latent user preferences that constrain future API argument selection. Key Principles: - Preference reasoning is holistic and non-decompositional. - Do NOT enumerate slots or list past actions. - Infer abstract constraints that explain multiple past decisions. - A valid preference must be actionable: it should rule in or rule out future API arguments. If evidence is insufficient, state uncertainty explicitly. ### Context (Accumulated Data) **Previous Belief**: {prev_implicit} **Dialogue History**: {full_dialogue} **API Calls**: {full_api_calls}	Generate Prompt ### Task: Infer Latent Preference as an Action Constraint Given the accumulated interaction history, infer the user's latent preference as a compact decision-level constraint. Guidelines: 1. Do NOT describe individual turns or list slot values. 2. Identify a unifying preference that explains repeated or consistent choices. 3. If multiple signals exist, abstract them into a higher-level constraint. 4. If contradictions exist, prioritize the most recent stable pattern. 5. If no stable preference can be inferred, explicitly say "insufficient evidence". ### Output Format (JSON) <pre>{ "reasoning": "One-paragraph explanation of the inferred abstraction and its supporting evidence", "implicit_pref": "A single-sentence constraint describing how future API arguments should be biased or restricted" }</pre> Refine Prompt ### Task: Refine Preference Based on Evidence Gaps Your previous preference abstraction was rejected. You must: - Remove any claim not directly supported by the logs. - Increase abstraction if details are over-specified. - Preserve only what consistently constrains action selection. Do NOT add new information. ### Input Draft Preference: "{previous_draft}" Verifier Feedback: "{feedback}" ### Output Format (JSON) <pre>{ "reasoning": "How unsupported details were removed or abstracted", "implicit_pref": "The revised, evidence-tight preference constraint" }</pre>	You are a Preference Verification Module. Your task is to judge whether the candidate preference is a valid latent constraint derived from the interaction logs. Evaluation Criteria: 1. Evidence Support: - Every claim must be supported by multiple or consistent signals. 2. Abstraction Quality: - Reject preferences that merely restate slot values or actions. - Prefer abstract constraints that generalize across domains. 3. Actionability: - The preference must constrain or bias future API argument selection. - If it cannot affect future actions, it is invalid. 4. Temporal Consistency: - If behavior changed, ensure the preference reflects the latest stable pattern. ### Evidence (Logs) **Dialogue**: {full_dialogue} **API Calls**: {full_api_calls} ### Candidate Preference to Verify {candidate_pref} ### Output Format (JSON) <pre>{ "valid": true/false, "feedback": "If false, specify whether the issue is over-specificity, hallucination, lack of abstraction, or non-actionability." }</pre>

Figure 9: Prompt templates for the PRefine generator and verifier. The generator proposes latent preference hypotheses as abstract, decision-level constraints from accumulated interaction history. The verifier evaluates each candidate against four validity conditions and provides structured feedback for refinement.

Base Prompting	Memory-based (PRefine, LangMem, Mem0, RAG) Prompt
[Instruction] You are a Personalized Preference Extraction Specialist. Your goal is to infer the user's "Abstract Preferences" based on their dialogue history and current utterance, strictly following the provided preference schema. [Task Definition] The user may not explicitly state all information in the current turn. You must deduce missing information by analyzing patterns in the Dialogue History. - "Repetitiveness": If a user frequently chose a specific value in the past, assume this is their preference. - "Cross-domain Consistency": Identify the user's "universal behavioral patterns or constraints" exhibited in previous interactions. If a direct preference is missing, "deduce" the current slot's value by applying these established patterns. [Reasoning Steps] 1. "Filter Context": Focus ONLY on the slots listed in the <code>`Target Preference Schema`</code>. Ignore transient slots like specific dates or times. 2. "Identify Patterns in History": - Scan the <code>`Dialogue History`</code> for the target slots. - Determine the most likely preference value based on frequency and similarity. 3. "Formulate Output": - Generate the final function call using the deduced information. Target Preference Schema: {preference_schema} Dialogue History: {dialogue_history} Current User Utterance: {user_utterance} Output Format: {{Domain}}{{slot_name}}="{{value}}", ...) Now produce the final Service API call:	You are a Personalized Preference Reasoning Agent. Your task is to generate the most appropriate Service API call for the current user utterance by integrating (1) Retrieved Long-term Memories and (2) Accumulated API Call History, strictly following the given schema. [Task Definition] The user may not explicitly state all information in the current turn. You must deduce missing information by analyzing inferred preferences in the Retrieved Memories and accumulated API call history. - "Repetitiveness": If a user frequently chose a specific value in the past, assume this is their preference. - "Cross-domain Consistency": Identify the user's "universal behavioral patterns or constraints" exhibited in previous interactions. If a direct preference is missing, "deduce" the current slot's value by applying these established patterns. [Reasoning Steps] 1. "Schema Filtering": - Identify the target domain from the <code>`Schema`</code>. - Consider ONLY the slots defined in the schema. 2. "Relevant Memories": - Infer the value by applying the user's stable behavioral patterns - Map these abstract constraints to the most appropriate schema-valid value. 3. "Formulate Output": - Create a slot ONLY when there is reasonable support from API history or memories. - Do NOT create empty slots - Do NOT hallucinate values or infer beyond the schema. Schema (Consider valid slots for the domain): {preference_schema} Relevant Memories (User Preferences & Constraints): {retrieved_memories} Current User Utterance: {user_utterance} Output Format: {{Domain}}{{slot_name}}="{{value}}", ...) Now produce ONLY the final Service API call:

Figure 10: Inference prompts used in our experiments. The base prompting template (left) instructs the LLM to infer user preferences and generate the final API call directly from dialogue history and the current query. The memory-retrieved template (right) is used by PRefine, LangMem, Mem0, and RAG, which condition on retrieved preference memories instead of full dialogue history.